

Sprawozdanie sk2

Komunikator internetowy

1. Opis projektu.

Część kliencka projektu została napisana w języku Python 3.12, a część serwerowa w języku C. GUI wykonane zostało przy użyciu biblioteki PyQt6. Projekt pisany był i testowany na maszynie wirtualnej Ubuntu 13.3.0 oraz serwerze polluks.

Dane użytkowników aplikacji posegregowane są według 3 głównych folderów:

1. **chats** – zawierający pliki tekstowe reprezentujące chaty użytkowników
2. **friends** – zawierający pliki tekstowe reprezentujące listę znajomych oraz zaproszeń do znajomych użytkownika, którego nickiem jest ów plik podpisany.
3. **users** – zawierający nazwy chatów użytkownika oraz ich identyfikatory (nazwy plików w folderze \chats)

```
├─ chats
│   ├── Gwiazdka-Rudolf.txt
│   ├── Błyskawica-Gwiazdka.txt
│   └── Błyskawica-Gwiazdka-Rudolf.txt
├─ friends
│   ├── Błyskawica.txt
│   ├── Gwiazdka.txt
│   └── Rudolf.txt
├─ users
│   ├── Błyskawica.txt
│   ├── Gwiazdka.txt
│   └── Rudolf.txt
├─ klient.py
├─ serwer.c
└─ users.txt
```

Główne klasy składające się na stronę klienta:

1. Klasy **signup** i **signin** - odpowiadają za procesy rejestracji i logowania użytkownika. Signup umożliwia rejestrację, waliduje dane i przekierowuje do ekranu logowania po pomyślnym zakończeniu. Signin pozwala na wprowadzenie danych logowania, ich walidację i przesłanie do serwera, a po pomyślnym logowaniu przekierowuje użytkownika do głównego menu aplikacji.
2. Klasa **main_menu** - obsługuje główne okno aplikacji czatu, umożliwiając użytkownikowi wybór chatu, wysyłanie wiadomości, wyświetlanie historii rozmów oraz przełączanie się do listy znajomych. Dzięki cyklicznemu odświeżaniu danych i komunikacji z serwerem w tle zapewnia aktualizowanie czatów i historii wiadomości.
3. Klasa **znajomi** - reprezentuje główne okno aplikacji do zarządzania znajomymi, umożliwiając użytkownikowi przeglądanie znajomych, zaproszeń oraz zarządzanie grupami. Obsługuje interakcje, takie jak akceptowanie i odrzucanie zaproszeń, dodawanie znajomych oraz tworzenie grup.

2. Opis komunikacji pomiędzy serwerem i klientem.

Komunikacja serwer – klient opiera się na zasadzie żądań i odpowiedzi. Klient po uruchomieniu łączy się z serwerem, a ten tworzy dla niego wątek do obsługi tych żądań. Każdy blok obsługujący żądanie po stronie serwera nie rozpoczyna pracy dopóki nie otrzyma pełnego zbioru argumentów wymaganych do wykonania tego zlecenia. Przy ustalonym połączeniu klient może wysyłać następujące żądania:

1. **Tworzenie konta** – weryfikuje poprawność danych, unikalność nicku i odsyła flagę wynikową.
2. **Logowanie** – weryfikuje poprawność danych i odsyła flagę wynikową.
3. **Strona główna** – przesyła użytkownikowi jego listę chatów.

4. **Wyświetlenie chatu** – przesyła użytkownikowi zawartość wskazanego przez niego chatu (którego jest członkiem).
5. **Znajomi** – przesyła użytkownikowi jego listę znajomych oraz zaproszeń do znajomych.
6. **Wyszukiwanie użytkowników** – szuka w bazie użytkowników nicku wskazanego przez interesanta.
7. **Zaproszenie użytkowników** – wysyła zaproszenie do znajomych wskazanemu użytkownikowi.
 - 7.1 **Akceptacja zaproszenia** – akceptuje wskazane zaproszenie do znajomych.
 - 7.2 **Odrzucenie zaproszenia** – odrzuca wskazane zaproszenie do znajomych.
8. **Stworzenie grupowego chatu** – tworzy grupowy chat ze wskazaną dwójką znajomych użytkownika.
9. **Zapisanie wiadomości** – zapisuje wysłaną przez użytkownika wiadomość na wskazanym chacie.
10. **Wyjście** – zamyka połączenie między klientem i serwerem, kończąc obsługę żądań.

3. Podsumowanie.

Identyfikatory chatów użytkowników są konkatencją ich nicków posortowaną rosnąco wg. ASCII, zapewniając eliminację tworzenia redundantnych chatów dla tych samych użytkowników.

Aktualność danych po stronie użytkownika zapewniana jest poprzez cykliczne odświeżanie – automatyczne wysyłanie żądań o numerach **3, 4, 5** z interwałem liczącym 2 sekundy.

W strukturze aplikacji wszystkie dostępy do plików prywatnych użytkowników są zabezpieczone przed możliwą próbą otwarcia pliku nieistniejącego. Plik **users.txt** oraz podfoldery przedstawione w punkcie 1. sprawozdania muszą zostać stworzone własnoręcznie. Współbieżny dostęp do plików jest zabezpieczony przez odpowiednie stosowanie mutexów w celu uniknięcia deadlocków.

Klasy techniczne po stronie klienta:

1. Klasa **worker** - umożliwia wykonywanie żądań w osobnym wątku, zapewniając współbieżność w aplikacji PyQt6. Obsługuje sygnały do przekazywania wyników, błędów, aktualizacji czatu i zakończenia pracy, co pozwala na płynne przetwarzanie operacji bez blokowania interfejsu użytkownika.
2. Klasa **Message** umożliwia serializację flagi do postaci bajtów, co jest przydatne przy przesyłaniu danych w protokołach komunikacyjnych.

Dokładny protokół komunikacji i obsługi żądań zawarty jest w repozytorium GIT projektu. Zawiera m.in. dokładniejszy opis funkcjonalności każdego z bloków wykonawczych, parametry wejściowe oraz parametry wyjściowe.

W trakcie implementacji aplikacji znaczącą trudność wprowadziły opóźnienia i anomalie w przesyłanych wiadomościach, uzależnione - jak się nam wydaje - od platformy sprzętowej. Przy zbyt szybkim odświeżaniu danych klienta, bądź wysyłaniu żądań zdarzało się iż wiadomości zawierały artefakty, były duplikowane, podmieniane oraz łączone, co znacznie utrudniało proces tworzenia aplikacji z uwagi na jej niską responsywność. Dla kontrastu problemów tych nie posiadaliśmy kiedy pisaliśmy terminalowy szkielet aplikacji na serwerze **polluks**, na którym niemożliwym było jednak stworzenie interfejsu graficznego strony klienckiej projektu.